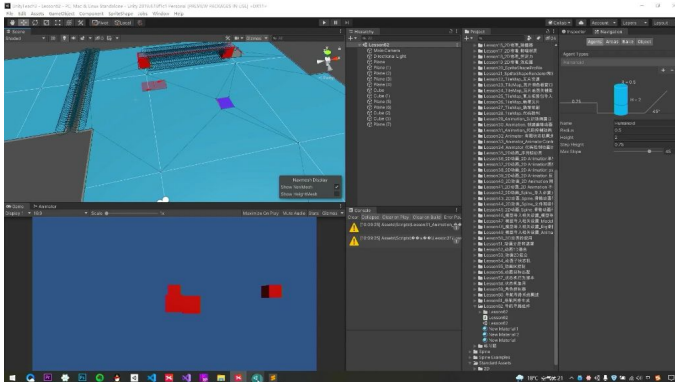


一、导航网格寻路组件 00:01

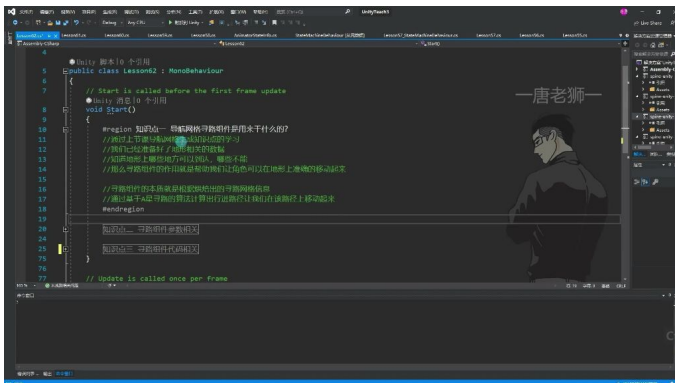
1. 场景介绍 00:52



- 场景组成：包含桥梁、桥洞、阶梯以及特殊寻路区域
- 特殊区域：中间部分标记为water区域，寻路消耗值为5
- 功能区域：包含两个分离平面用于演示跳跃功能

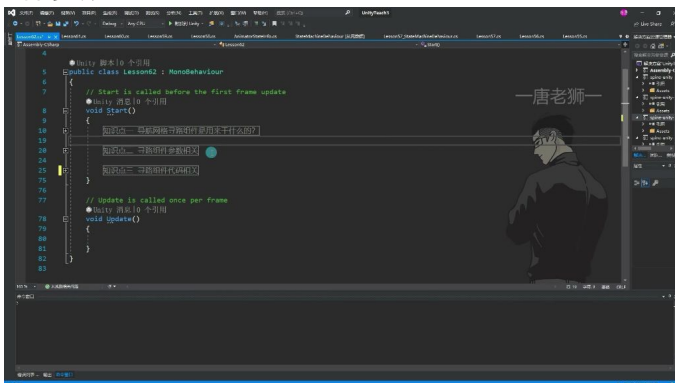
2. 导航网格寻路组件 01:24

1) 组件功能



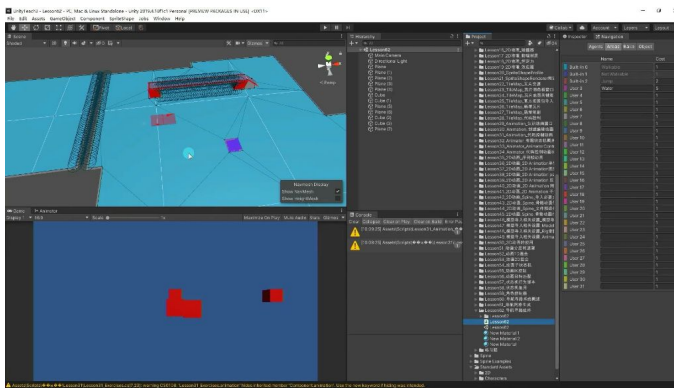
- 核心作用：基于烘焙好的导航网格数据，使游戏对象能在场景中自动寻路
- 工作原理：采用A*寻路算法计算最优路径，实现障碍物规避
- 数据依赖：需要预先通过导航网格生成组件准备地形可通行数据

2) 组件参数



- 消耗值影响：不同区域可设置不同移动消耗值（如water区域消耗值为5）
- 路径计算：消耗值越高，算法越倾向于避开该区域
- 特殊功能：支持跳跃等特殊移动方式的参数配置

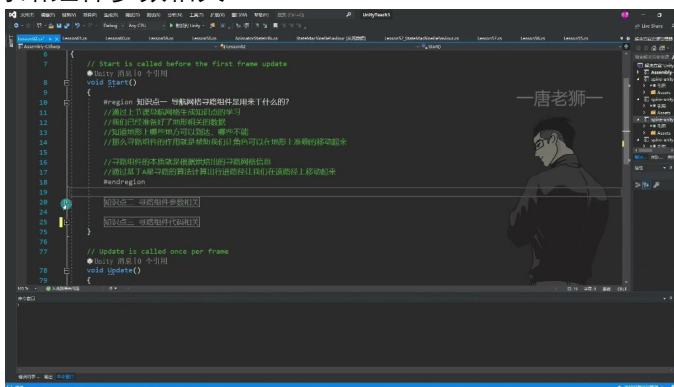
3) 代码控制



-
- **路径生成**：自动计算起点到终点的最优路径
- **移动控制**：角色沿计算路径自动移动
- **动态避障**：实时响应场景中的障碍物变化

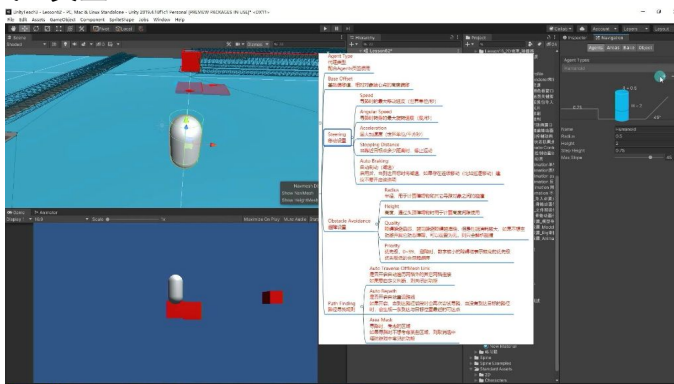
3. 知识点二 02:22

1) 寻路组件参数相关 02:25

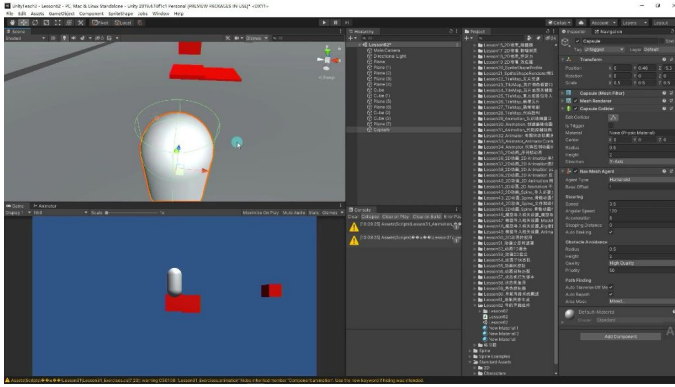


-
- **组件作用**：根据烘焙出的寻路网格信息，通过A*寻路算法计算行进路径，使角色能在地形上准确移动
- **本质原理**：基于导航网格数据和A*算法实现路径计算

2) 代理类型 04:35

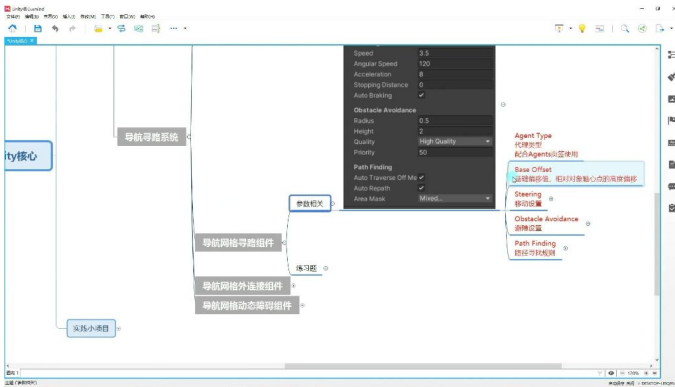


- - **配合使用**：需与导航面板中的Agents页签配合使用
 - **默认设置**：默认为人形代理(Humanoid)，一般无需修改
 - **数据关联**：代理参数会根据Agents页签中的预设数据进行自动配置
- ### 3) 基础偏移值 04:57



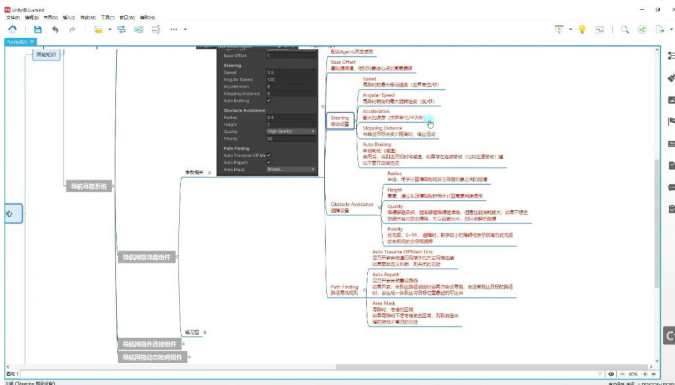
-
- 作用：调整寻路圆柱体相对于对象轴心点的高度偏移
- 调整效果：修改该值会改变寻路圆柱体的垂直位置
- 使用建议：当角色模型与轴心点高度不匹配时需要调整，简单模型(如胶囊体)可保持默认

4) 移动设置 05:23



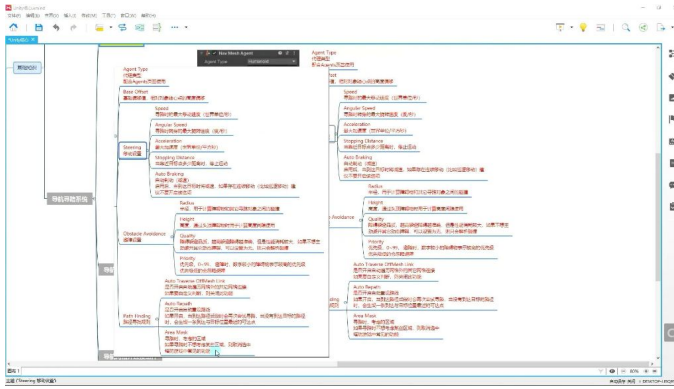
-
- 移动速度：最大移动速度(单位：米/秒)
- 旋转速度：转身最大旋转速度(单位：度/秒)
- 加速度：从零加速到最大速度的加速度(单位：米/平方秒)
- 停止距离：距离目标点多远时停止移动(单位：米)
- 自动制动：
 - 启用时会在接近目标点减速
 - 连续移动场景(如巡逻)建议禁用该选项

5) 避障设置 07:17



-
- 半径：决定寻路圆柱体的宽度，影响避障计算范围
- 高度：决定寻路圆柱体的高度，用于计算高度间隙
- 品质：避障精确度等级(无/低/中/高)，品质越高性能消耗越大
- 优先级：0-99数值，数值越小优先级越高，优先级低的会忽略避障

6) 路径寻找规则 08:38

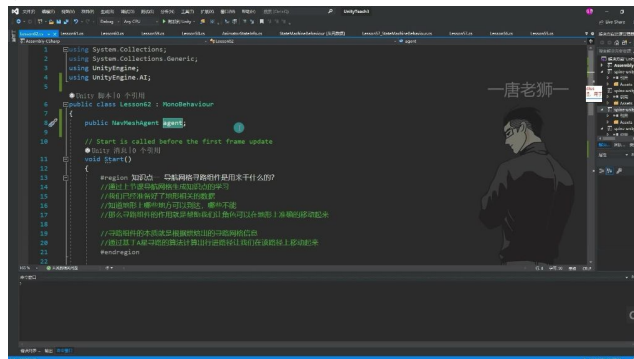


-
- 网格外连接：是否自动遍历网格外的其他网络连接
- 自动重设路线：
 - 启用时会在路径末端重新尝试寻路
 - 无法到达目标时会寻找最近可达点
 - 建议保持启用状态
- 区域遮罩：
 - 可选择寻路时考虑的区域类型
 - 塔防游戏常用功能(如设置某些单位不走水路)
 - 对应导航烘焙窗口中的区域设置

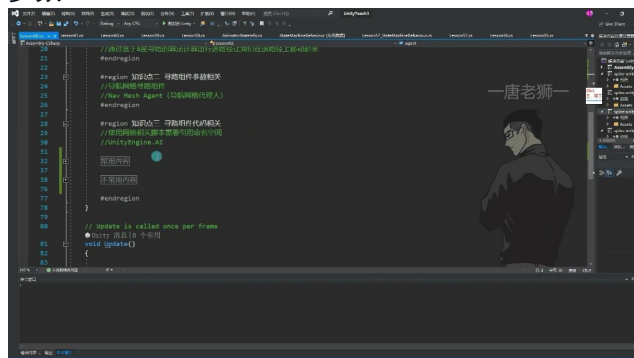
4. 知识点三 10:29

1) 导航网格寻路组件 10:34

- 组件功能

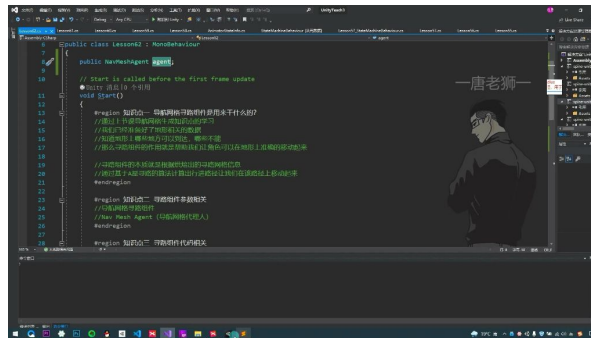


-
- 核心功能：根据烘焙出的寻路网格信息，通过基于A星寻路的算法计算出行进路径，使角色能在地形上准确移动
- 使用前提：需要先通过导航网格生成知识点准备好地形数据，明确可到达区域和障碍区域
- 组件参数

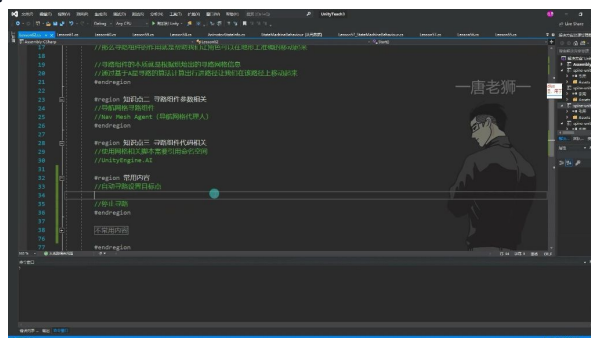


-
- 组件名称：Nav Mesh Agent（导航网格代理人）

- **参数配置**: 包含移动速度、转向速度、停止距离等可调参数, 用于控制代理人的寻路行为
- **代码实现**
 - **基本设置**

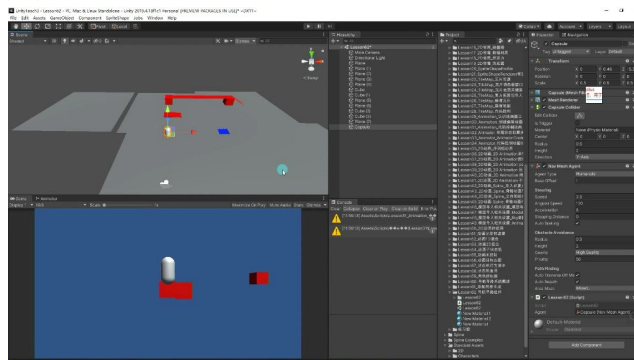


- **命名空间**: 必须引用UnityEngine.AI命名空间才能使用导航相关脚本
- **组件声明**: 通过public NavMeshAgent agent;声明导航代理变量, 需挂载到游戏对象上
- **常用API**



- **自动寻路**: 使用agent.SetDestination(目标点)方法设置移动目标点
- **停止寻路**: 通过agent.isStopped = true停止当前寻路, 设为false可恢复移动
- **组件关联**: 在Inspector窗口将声明的public变量与游戏对象上的NavMeshAgent组件关联

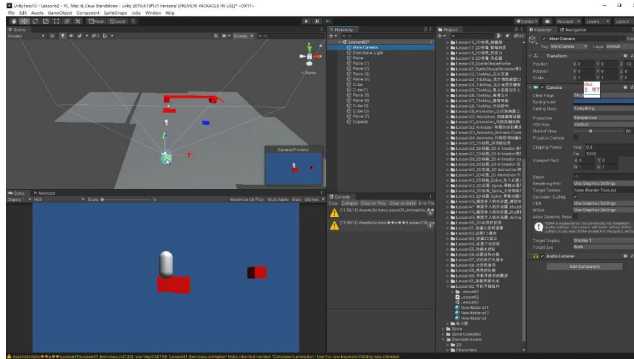
● 实践应用



- **实现步骤**:
 - 创建脚本并声明NavMeshAgent变量
 - 将脚本挂载到需要移动的游戏对象上
 - 在Inspector中关联NavMeshAgent组件
 - 调用SetDestination方法设置目标位置
 - 通过isStopped控制移动启停

2) 应用案例 12:47

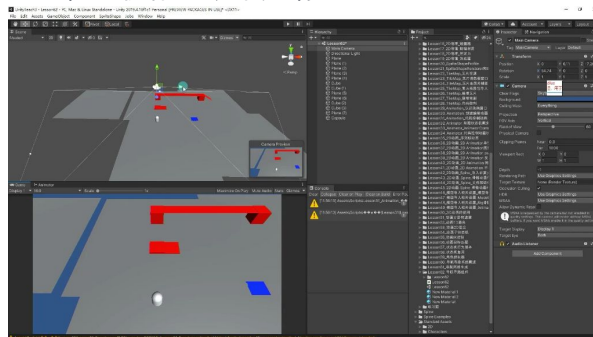
- **点击移动功能实现**



-
-

摄像机调整：

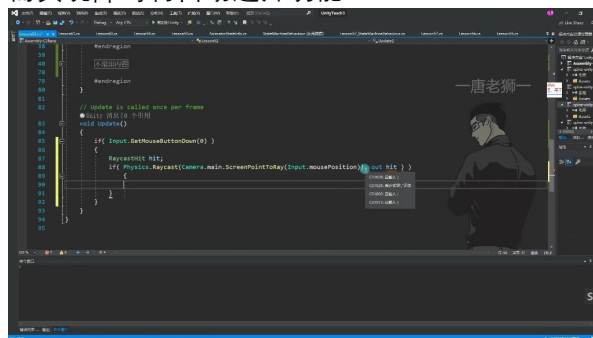
- 将摄像机高度上抬，X轴旋转形成俯视视角
- 调整位置使俯视效果更清晰



-

功能需求：

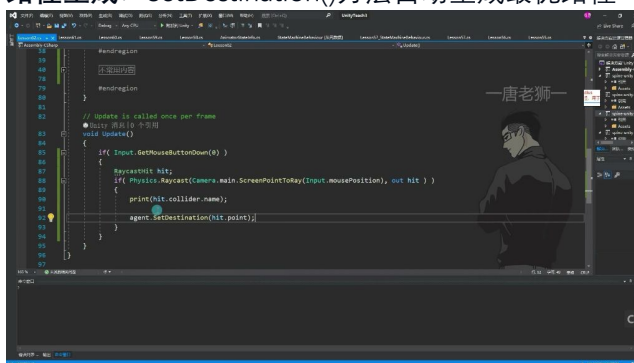
- 点击Game窗口任意位置，游戏对象自动寻路至目标点
- 需实现障碍物自动避开功能



-

核心代码：

- 射线检测：从屏幕点击位置发射射线进行碰撞检测
- 路径生成：SetDestination()方法自动生成最优路径

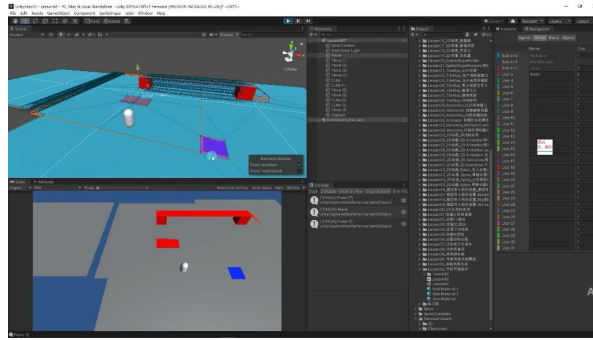


-
-

关键参数：

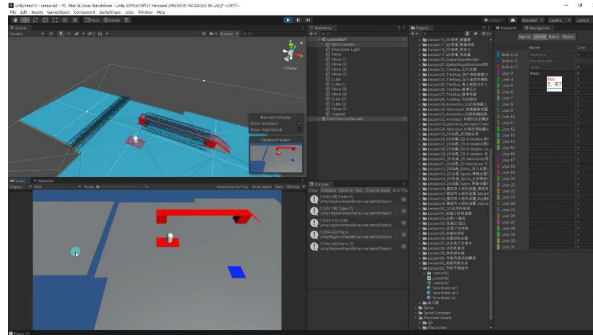
- Speed：移动速度（世界单位/秒），默认3.5
- Angular Speed：转身速度（度/秒），默认120

- **Acceleration:** 加速度（世界单位/平方秒），默认8
- **Stopping Distance:** 停止距离（世界单位），默认0



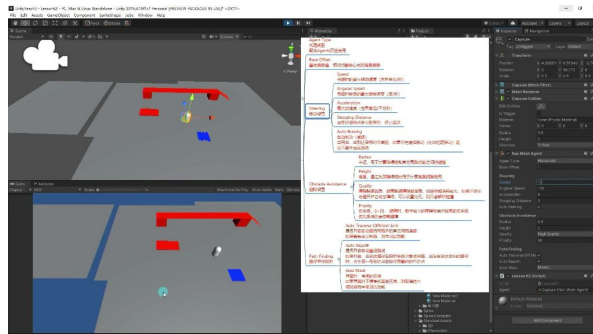
○ **寻路区域:**

- 不同区域可设置不同移动消耗值（如水域消耗设为5）
- 消耗值越高，AI越倾向于绕行
- 通过Navigation窗口设置区域类型和消耗值



○ **连接点设置:**

- 相连平面需同时勾选Navigation Static
- 未同时勾选会导致无法跳跃
- 修改后需重新烘焙导航网格



○ **调试经验:**

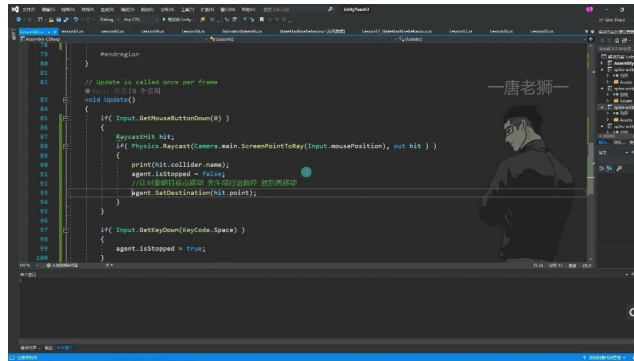
- 移动速度与加速度配合：高加速度（如100）可实现瞬时加速
- 转身速度影响角色朝向调整：建议设为360度/秒
- 停止距离设为0可使对象精确到达目标点

○ **避障原理:**

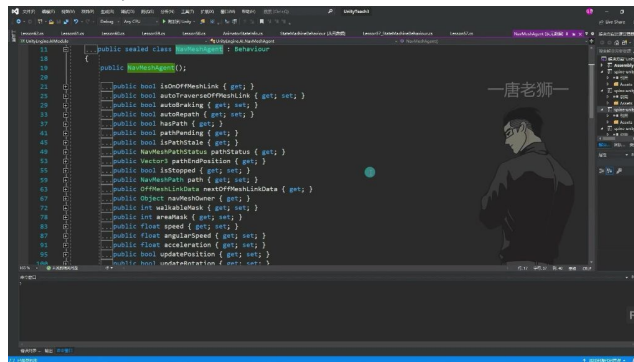
- 通过半径（Radius）和高度（Height）计算碰撞
- 优先权（Priority）决定避让顺序（0-99，数值越小优先级越高）
- 自动制动（Auto Braking）影响到达目标时的减速行为

3) 不常用内容 25:47

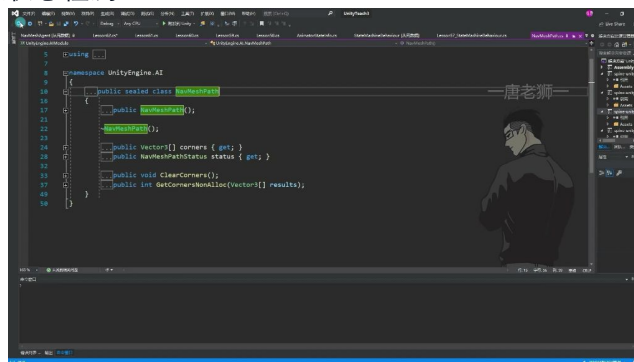
- 示例动画演示停止寻路 26:12



- 停止移动实现：通过设置agent.isStopped = true可立即停止当前寻路
- 恢复移动要点：必须将isStopped重置为false才能恢复寻路功能，否则会永久停止
- 典型应用场景：空格键触发停止 (Input.GetKeyDown(KeyCode.Space))
- 导航组件关键变量

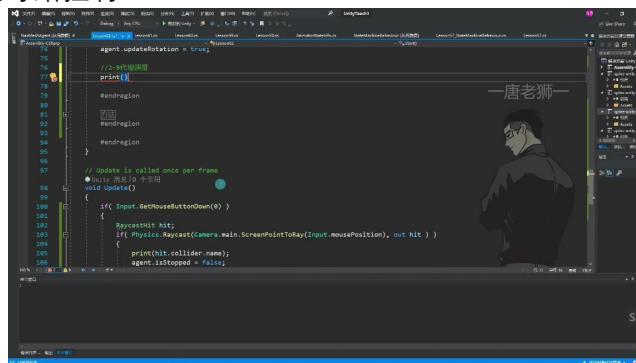


- 运动参数：
 - speed：移动速度（单位：米/秒）
 - acceleration：加速度参数
 - angularSpeed：角速度（旋转速度）
- 状态参数：
 - hasPath：判断当前是否有有效路径
 - destination：获取/设置目标点坐标
 - isStopped：获取/设置停止状态
 - path：获取当前路径点序列（NavMeshPath对象）
- 路径状态检测

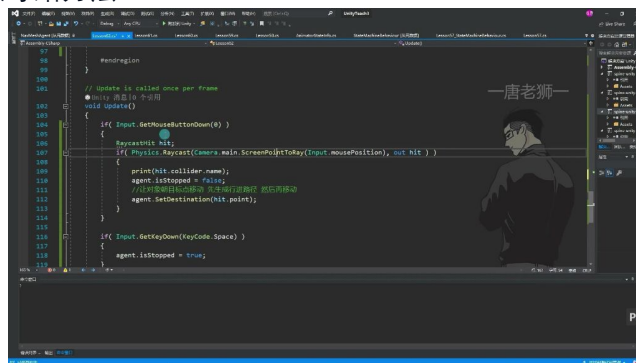


- 路径状态类型：
 - PathComplete：0（完全成功）
 - PathPartial：1（部分成功）
 - PathInvalid：2（完全无效）
- 实时检测方法：
 - pathPending：检测路径是否在计算中

- pathStatus: 获取当前路径状态枚举值
- 手动寻路控制



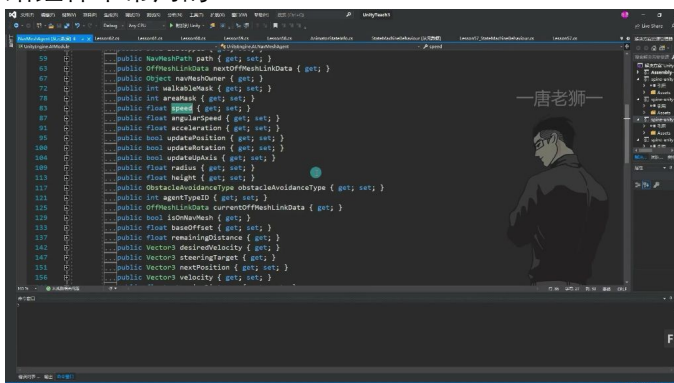
-
- 位置更新控制:
 - updatePosition: 设为false可禁用自动位置更新
 - updateRotation: 设为false可禁用自动旋转
- 速度检测:
 - velocity: 获取当前速度向量 (Vector3类型)
 - 停止判断: agent.velocity == Vector3.zero
- 高级寻路方法



-
- 路径计算:
- 路径操作:
 - SetPath(): 应用预计算的路径
 - ResetPath(): 清除当前路径
 - Warp(): 瞬移到指定位置

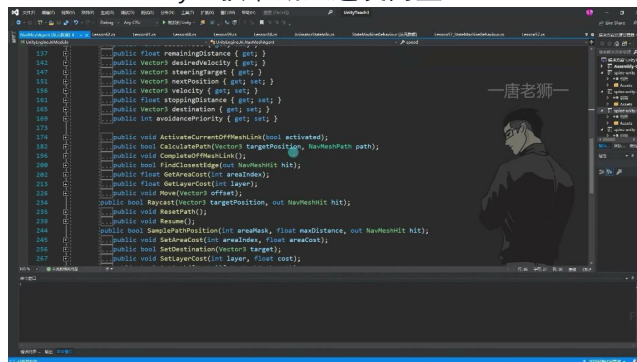
二、寻路组件 36:13

1. 寻路组件不常用的API 36:33



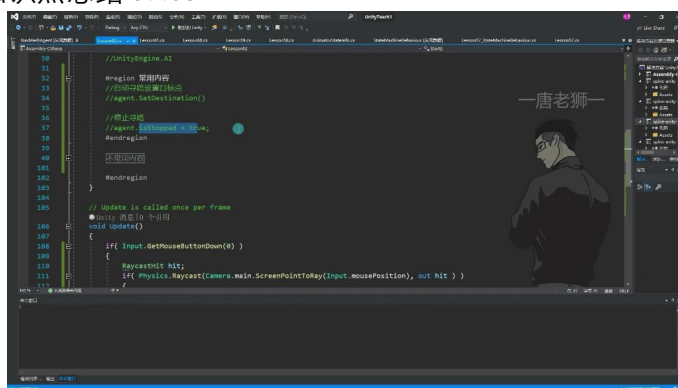
-
- 高级功能API: 包括路径状态检查、位置更新控制等, 适用于特殊需求开发
 - pathStatus: 获取当前路径状态
 - updatePosition: 控制是否更新代理位置

- updateRotation: 控制是否更新代理角度
- velocity: 获取代理当前移动速度
- remainingDistance: 获取剩余距离
- desiredVelocity: 获取期望速度向量

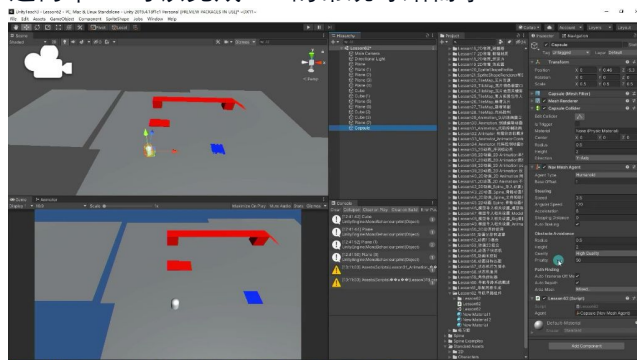


- 方法类API:
 - ActivateCurrentOffMeshLink(): 激活当前网格外链接
 - CalculatePath(): 计算路径但不移动
 - FindClosestEdge(): 寻找最近边缘点
 - SetAreaCost(): 设置区域移动成本
 - SetLayerCost(): 设置层级移动成本
 - Warp(): 瞬移到指定位置
- 使用建议: 常规游戏开发只需掌握基础API即可满足需求, 这些高级API主要用于特殊场景下的路径控制优化

2. 知识点总结 37:03



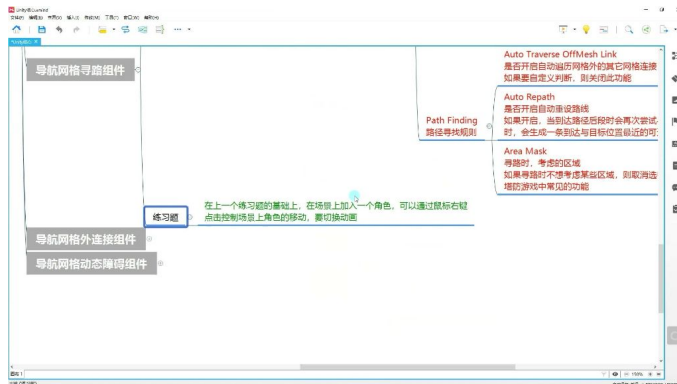
- 核心功能:
 - SetDestination(): 设置移动目标点
 - isStopped: 停止/恢复寻路
 - 这两个API可以完成90%的常规寻路需求



- 重要参数:
 - Speed: 移动速度

- Angular Speed: 转向速度
- Acceleration: 加速度
- Radius: 代理半径
- Height: 代理高度
- Obstacle Avoidance: 避障类型
- 开发要点:
 - 必须配合导航网格烘焙数据使用
 - 需要理解区域掩码(Area Mask)的作用
 - 常规项目无需使用高级API
 - 特殊需求可查阅官方文档扩展功能

3. 练习题: 角色寻路控制



- 题目解析
 - 实现角色通过鼠标点击移动
 - 需要处理寻路动画切换
 - 结合上节课的导航网格烘焙
 - 答案将在下节课讲解

三、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
导航网格寻路组件的作用	根据烘焙的导航网格信息, 通过A*算法计算路径, 实现游戏对象自动避障移动	组件与导航网格生成的关联性	★★
寻路组件参数	- 移动设置: 速度、加速度、停止距离; - 避障设置: 半径、优先级; - 路径规则: 区域遮罩、自动重设路线	区域遮罩对塔防游戏兵种路径的影响	★★★
代码实现自动寻路	NavMeshAgent.SetDestination()设置目标点; isStopped控制寻路启停	停止后需重置isStopped才能重新寻路	★★
路径动态控制	获取路径点序列 (NavMeshPath)、手动更新位置/旋转 (updatePosition)	策略游戏中路径预览功能的实现逻辑	★★★★
寻路区域与消耗	不同区域 (如water) 设置寻路消耗值, 影响路径选择优先级	消耗值≠遮罩: 高消耗绕行 vs 遮罩禁止通行	★★★

跳跃连接点	需确保相连平面均匀选 Navigation Static才能双向跳跃	单向跳跃失效 的排查方法	★ ★
-------	---------------------------------------	-----------------	-----